

```

# Tic-Tac-Toe AI Agent using Minimax

board = [' ' for _ in range(9)]

def print_board(board):
    print(f"\n {board[0]} | {board[1]} | {board[2]} ")
    print("-----")
    print(f" {board[3]} | {board[4]} | {board[5]} ")
    print("-----")
    print(f" {board[6]} | {board[7]} | {board[8]} \n")

def check_winner(b, player):
    win_conditions = [
        [0,1,2], [3,4,5], [6,7,8], # Rows
        [0,3,6], [1,4,7], [2,5,8], # Columns
        [0,4,8], [2,4,6]           # Diagonals
    ]
    return any(all(b[i] == player for i in cond) for cond in
win_conditions)

def is_full(b):
    return ' ' not in b

def minimax(b, is_maximizing):
    if check_winner(b, 'O'): return 1
    if check_winner(b, 'X'): return -1
    if is_full(b): return 0

    if is_maximizing:
        best_score = -float('inf')
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'O'
                score = minimax(b, False)
                b[i] = ' '
                best_score = max(score, best_score)
        return best_score
    else:
        best_score = float('inf')
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'X'
                score = minimax(b, True)
                b[i] = ' '
                best_score = min(score, best_score)
        return best_score

```

```

def ai_move(b):
    best_score = -float('inf')
    move = None
    for i in range(9):
        if b[i] == ' ':
            b[i] = 'O'
            score = minimax(b, False)
            b[i] = ' '
            if score > best_score:
                best_score = score
                move = i
    b[move] = 'O'

# Game Loop
print("You are X, AI is O.")
print_board(board)

while True:
    # Human Move
    pos = int(input("Enter move position (0-8): "))
    if board[pos] == ' ':
        board[pos] = 'X'
    else:
        print("Invalid move! Try again.")
        continue

    if check_winner(board, 'X'):
        print_board(board)
        print("You win!")
        break
    if is_full(board):
        print_board(board)
        print("It's a tie!")
        break

    # AI Move
    ai_move(board)
    print("AI played:")
    print_board(board)

    if check_winner(board, 'O'):
        print("AI wins!")
        break
    if is_full(board):
        print("It's a tie!")
        break

```

```

# Vacuum Cleaner AI Agent

def vacuum_cleaner_agent():
    # Goal State: Both locations A and B must be Clean ('0')
    goal_state = {'A': '0', 'B': '0'}
    cost = 0

    # User inputs for initial environment configuration
    location = input("Enter starting location of Vacuum Agent (A or B): ").strip().upper()
    status_current = input(f"Enter status of Location {location} (1 for Dirty, 0 for Clean): ").strip()

    other_location = 'B' if location == 'A' else 'A'
    status_other = input(f"Enter status of Location {other_location} (1 for Dirty, 0 for Clean): ").strip()

    # Environment status dictionary
    environment = {
        location: status_current,
        other_location: status_other
    }

    print("\n--- Simulation Started ---")
    print(f"Initial State: {environment}")

    # Process first location
    if environment[location] == '1':
        print(f"Location {location} is Dirty.")
        print(f"Action: SUCK (Cleaning Location {location})")
        environment[location] = '0'
        cost += 1
    else:
        print(f"Location {location} is already Clean.")

    # Move to the adjacent location
    if location == 'A':
        print("Action: MOVE RIGHT (A -> B)")
        location = 'B'
        cost += 1
    else:
        print("Action: MOVE LEFT (B -> A)")
        location = 'A'
        cost += 1

    # Process second location
    if environment[location] == '1':

```

```
    print(f"Location {location} is Dirty.")
    print(f"Action: SUCK (Cleaning Location {location})")
    environment[location] = '0'
    cost += 1
else:
    print(f"Location {location} is already Clean.")

# Check goal state
print("\n--- Simulation Summary ---")
print(f"Final Environment State: {environment}")
print(f"Total Cost (Steps Taken): {cost}")

if environment == goal_state:
    print("Result: Goal achieved! Both locations are clean.")

if __name__ == "__main__":
    vacuum_cleaner_agent()
```